Faculty of Engineering and Technology

Electrical and Computer Engineering Department

Database Management systems

COMP333

Prepared by:

Alzahra Nassif **1220168**

Haneen Qaisi **1220201**

Instructor: Bassem Sayrafi

Section: 2

Date: 18/1/2026

Project Scope and Description

This project aims to build a Library Management and Sales Database System for AlSarisi Library in Ramallah. The goal is to help the library keep track of its products (books, notebooks, stationery), manage purchases from suppliers, and organize daily sales to customers through multiple branches.

The system will update the stock automatically whenever a sale or a purchase is recorded, so employees can always know the real quantity available for each product. It will also provide simple reports, like the most sold items, products running low, and total sales in a given period.

Besides stock control, the system will also save information about customers, staff, and suppliers. By connecting all this data together, the library can improve how it works every day, avoid mistakes in counting inventory, and make better decisions about what to order.

Client Information:

Client Name: Al-Sarisi Library

Location: Ramallah, Palestine

Industry: Retail & Educational Supplies (books, notebooks, stationery, and school materials)

Key Requirements: Sales tracking, purchase management, warehouse management, supplier records, staff and customer management .

Contact information:

Phone Number: 02 298 7479

Email: info@alsarisi.com

Technology Stack:

Backend: Python-flask

Frontend: HTML, CSS, JS

Database: MySQL

Main Tables:

1. **Customer**: Contains customer information such as customer_id, first_name, last_name, email, phone, address.
2. **Category**: Stores category details such as category_id, name, description.
3. **Product**: Holds product details such as product_name, product_id, category_id, supplier_id, sku, description, unit_price,purchase_price
4. **Order**: Stores general order information such as order_id, customer_id, order_date, branch_id ,staff_id
5. **Order_Item**: An intermediary table between Order Table and Product

   Table, containing order_item_id, order_id, product_id, quantity, unit_price, warehouse_id.

6. **Branch**: Stores branch details including branch_id, name, city, address, phone.
7. **Staff:** Contains staff details such as staff_id, first_name, last_name, position, salary, branch_id, email, phone.
8. **Supplier**: Stores supplier details including supplier_id, supplier_name, contact_name, contact_phone, email, address.
9. **Warehouse**: Contains warehouse data such as warehouse_id, location, capacity, branch_id, name.
10. **Inventory**: Tracks stock levels per warehouse with inventory_id, product_id, warehouse_id, stock_quantity, last_updated.
11. **Purchase**: Represents the supplier invoice for purchased products. Each purchase record corresponds to one invoice received from a supplier. It contains purchase_id, supplier_id, branch_id, purchase_date, invoice_number, and total_amount, staff_id

12. **PurchaseDetail**: Contains the detailed items listed in each supplier invoice, including purchaseDetail_id, purchase_id, warehouse_id, product_id, quantity, unit_cost.
13. **Bill**: Stores basic invoice information for a customer order, such as bill_id, order_id, bill_date, total_amount, payment_method, payment_status

14. **StockMovement**:Tracks inventory movements including movement_id, movement_date, movement_type, product_id, warehouse_id, staff_id, qty_change.

Sample Queries

1. Retrieve details of all branches located in a specific city.
2. Retrieve the names, positions, and salaries of staff members at a specific branch, sorted by staff name.
3. Retrieve the names of all Managers at each branch, sorted by branch number.
4. Retrieve the title, category, and availability of all products at a specified branch, sorted by category.
5. Retrieve the title, category, and availability of all products supplied by a specific supplier at a specified branch, sorted by the warehouse entry date.
6. Retrieve the total number of products in each category at a given branch, sorted by supplier.
7. Display the total sales revenue for each branch in a given month.
8. Retrieve the best-selling product (by sales quantity) in a specific category.
9. Retrieve the top 3 bestselling products in the last month
10. Retrieve current inventory status for a specific product.
11. List all products that need restocking because their availability is below a threshold.
12. Retrieve supplier contact information for a specific product.
13. Retrieve a monthly sales summary for each branch.
14. Retrieve the branch with the highest sales revenue in the last month
15. Retrieve the list of products that have not been sold in the last 10 days.
16. Retrieve the list of all registered suppliers along with their contact details.
17. Retrieve the warehouse with the lowest stock levels.

Relationships

- **Branch -- Staff (One-to-Many): -** employs-

  A branch employs multiple staff members and a staff member is employed by exactly one branch.

- **Branch --Order (One-to-Many):** - processes-

  A branch processes multiple orders and an order is processed at exactly one branch.

- **Branch – Warehouse (One-to-One):** -Owns-

  Each branch owns one warehouse, and warehouse belongs to exactly one branch.

- **Supplier – Product (One-to-Many): -** Supplies-

  A supplier can provide many products, but each product has one supplier.

- **Category -- Product (One-to-Many): -** classifies-

  A category classifies multiple products and a product is classified under exactly one category.

- **Order -- Order_Item (One-to-Many): -**Contains-**:**

  Each order contains multiple order items, but each order item belongs to exactly one order.

- **Product -- Order_Item (One-to-Many): -** classifies-

  A product may appear in many order items, but each order item refers to one product.

- **Warehouse – Order_Item (One-to-Many): -** Handles-

  Each order item reduces stock from one warehouse, and a warehouse may fulfill many order items.

- **Supplier -- Purchase (One-to-Many): -**Invoices-

  A supplier provides multiple purchases and a purchase is linked to one supplier.

- **Branch – Purchase (One-to-Many): -** issues-

  Each branch creates its own purchase orders, but a purchase is tied to one branch.

- **Purchase – PurchaseDetail (One-to-Many): -**includes-

  A purchase invoice contains multiple purchase details, but each detail belongs to one purchase.

- **Product -- PurchaseDetail (One-to-Many): -** Purchased-

  A product may appear in many purchase details, but each purchase detail refers to one product.

- **Warehouse -- Purchase Detail (One-to-Many): -** Receives-

  A warehouse receives multiple purchases detail and a purchase detail is received at exactly one warehouse.

- **Product – Inventory- Warehouse (Ternary Relationship):**
  - stored_in -

  The stored_in relationship represents a ternary relationship between Product, Warehouse, and Inventory. It indicates that a specific product is stored in a specific warehouse with an associated inventory record.

  .

- **Warehouse -- Inventory (One-to-Many): -**Generates-**:**

  A warehouse contains multiple inventory records and inventory record is housed in exactly one warehouse.

- **Order--Bill(One-to-One) -**Generates**-:**

  Each order generates one sales invoice, and each invoice belongs to exactly one order.

- **Staff-Purchase(One-to-Many):-**has**-**

  Each staff member may be responsible for many purchases, while each purchase is recorded by one staff member.

- **Customer - Staff - Order (Handles_Sale)** :(Ternary Relationship): The Handles_Sale relationship represents a ternary relationship between Customer, Staff, and Order. It indicates that a staff member handles a sale (order) for a specific customer.

- **Staff -StockMovement (Performs) (One-to-Many):**

  A staff member performs multiple stock movements, and each stock movement is performed by one staff member.

- **Product - StockMovement (Has) (One-to-Many):**

  A product can have multiple stock movement records, and each stock movement record is associated with one product.

- **Warehouse - StockMovement (Occurs_In) (One-to-Many):**

  A warehouse is the location where multiple stock movements occur, and each stock movement occurs in one warehouse.

Normalization (1NF → 3NF)

- **First Normal Form (1NF)**

All relations satisfy 1NF because:

👌 All attributes are atomic (single-valued).

👌 There are no repeating groups in a single table.

👌 Each table has a defined primary key.

- **Second Normal Form (2NF)**

2NF is concerned with **partial dependency**, which only occurs when a table has a **composite key** and a non-key attribute depends on part of that key.
In our design:

👌 tables mainly use **single-attribute primary keys** (IDs) , so partial dependency does not occur.

👌 Detail/transaction tables (Order_Item, PurchaseDetail, Inventory) use surrogate IDs

👌 **Result:** All our tables satisfy **2NF**.

- **Third Normal Form (3NF)**

3NF eliminates **transitive dependencies (**when a non-key attribute depends on another non-key attribute**)** ,and all tables satisfy 3NF because:

👌 Non-key attributes depend on the key only.

👌 No table stores attributes that depend on another non-key attribute (i.e., transitive dependencies are avoided by separating entities like Supplier, Category, Branch, etc.)?????????

Step-by-Step Normalization

📄 Customer Table

Primary Key: customer_id
Non-Key Attributes: first_name, last_name, email, phone, address
Functional Dependency:

- customer_id → first_name, last_name, email, phone, address

1NF: ✓ All attributes are atomic
2NF: ✓ Single-attribute primary key
3NF: ✓ No transitive dependencies

📄 Category Table

Primary Key: category_id
Non-Key Attributes: name, description
Functional Dependency:

- category_id → name, description

1NF: ✓ All attributes are atomic
2NF: ✓ Single-attribute primary key
3NF: ✓ No transitive dependencies

📄 Supplier Table

Primary Key: supplier_id
Non-Key Attributes: supplier_name, contact_name, contact_phone, email, address
Functional Dependency:

- supplier_id → supplier_name, contact_name, contact_phone, email, address

1NF: ✓ 2NF: ✓ 3NF: ✓

📄 Product Table

Primary Key: product_id
Non-Key Attributes: product_name, category_id, supplier_id, sku, description, unit_price, purchase_price
Functional Dependency:

- product_id → product_name, category_id, supplier_id, sku, description, unit_price, purchase_price

1NF: ✓ 2NF: ✓3 NF: ✓

Branch Table

Primary Key: branch_id
Non-Key Attributes: name, city, address, phone
Functional Dependencies:

- branch_id → name, city, address, phone

1NF: ✓ 2NF: ✓3NF: ✓

Staff Table

Primary Key: staff_id
Non-Key Attributes: first_name, last_name, position, salary, branch_id, email, phone
Functional Dependencies:

- staff_id → first_name, last_name, position, salary, branch_id, email, phone

1NF: ✓2NF: ✓3NF: ✓

Warehouse Table

Primary Key: warehouse_id
Non-Key Attributes: location, capacity, branch_id, manager_staff_id, name
Functional Dependencies:

- warehouse_id → location, capacity, branch_id, manager_staff_id, name

1NF: ✓2NF: ✓3NF: ✓

Inventory Table

Primary Key: inventory_id
Non-Key Attributes: product_id, warehouse_id, stock_quantity, last_updated
Functional Dependencies:

- inventory_id → product_id, warehouse_id, stock_quantity, last_updated

1NF: ✓ 2NF: ✓3NF: ✓

Order Table

Primary Key: order_id
Non-Key Attributes: customer_id, order_date, branch_id, staff_id
Functional Dependencies:

- order_id → customer_id, order_date, branch_id, staff_id

1NF: ✓ 2NF: ✓3NF: ✓

Order_Item Table

Primary Key: order_item_id
Non-Key Attributes: order_id, product_id, quantity, unit_price, warehouse_id
Functional Dependencies:

- order_item_id → order_id, product_id, quantity, unit_price, warehouse_id

1NF: ✓ 2NF: ✓3NF: ✓

Purchase Table

Primary Key: purchase_id
Non-Key Attributes: supplier_id, branch_id, purchase_date,

total_amount, invoice_number, staff_id

Functional Dependencies:

- purchase_id→supplier_id, branch_id, purchase_date, total_amount, invoice_number, staff_id

1NF: ✓ 2NF: ✓3NF: ✓

PurchaseDetail Table

Primary Key: purchaseDetail_id

Non-Key Attributes: purchase_id, warehouse_id, product_id, quantity, unit_cost

Functional Dependencies:

- purchaseDetail_id →purchase_id, warehouse_id, product_id, quantity, unit_cost

1NF: ✓ 2NF: ✓3NF: ✓

Bill Table

Primary Key: bill_id

Non-Key Attributes: order_id, bill_date, total_amount, payment_method, payment_status

Functional Dependencies:

- bill_id→order_id, bill_date, total_amount, payment_method, payment_status

1NF: ✓ 2NF: ✓3NF: ✓

StockMovement Table

Primary Key: movement_id

Non-Key Attributes: movement_date, movement_type, product_id, warehouse_id, staff_id, qty_change

Functional Dependencies:

- movement_id→movement_date,movement_type, product_id, warehouse_id, staff_id, qty_change

1NF: ✓ 2NF: ✓3NF: ✓